SILVA: Parallel Spatial Index Library with Versioned Access (technical report)

Bo Huang
UC Riverside
bo.huang@email.ucr.edu

Ahmed Eldawy
UC Riverside
eldawy@ucr.edu

Yan Gu
UC Riverside
ygu@cs.ucr.edu

Yihan Sun
UC Riverside
yihans@cs.ucr.edu

Abstract

Spatial datasets are now generated at an ever-growing scale, featuring massive sizes and high-frequency modifications. Modern applications must efficiently manage this dynamic data while also maintaining historical and hypothetical versions for comprehensive analytics. Ideally, a spatial index should efficiently incorporate updates with parallelism while preserving historical states, and support version-aware queries as efficient as single-versioned indexes. However, no existing spatial index supports both parallel updates and comprehensive version access. This paper presents SILVA, a library of main-memory spatial indexes designed for versioned access to highly dynamic spatial data. We formalize the problem of multi-version spatial data management and propose multiple indexes that build on state-of-the-art functional data structures to support immutable versions with atomic batch updates. SILVA is highly parallel and space-efficient, enabling seamless spatial queries across versions. Experiments on synthetic and real-world datasets show that SILVA outperforms state-of-the-art indexes by up to two orders of magnitude, peaking at 112 million updates per second, while reducing memory usage by 70%.

CCS Concepts

• **Information systems → Parallel and distributed DBMSs**.

Keywords

Versioned Access, Spatial Data, Functional Data Structures

ACM Reference Format:

Bo Huang, Ahmed Eldawy, Yan Gu, and Yihan Sun. 2026. SILVA: Parallel Spatial Index Library with Versioned Access (technical report) . In *Proceedings of The 34th ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL '26).* ACM, New York, NY, USA, 16 pages.

1 Introduction

With the rapid advancement of geospatial technologies, spatial data are generated and collected at an ever-growing scale, and are updated and shared more frequently than ever (e.g., daily crime reports [18], traffic accident records [41], and demographic shifts [19]). As a result, modern applications such as collaborative mapping and agentic systems increasingly rely on analyzing how spatial data evolve over time. For these use cases, querying the "here and now" no longer suffices; instead, these systems demand the capability to traverse data timelines or even explore a hypothetical "multiverse." Examples include retrieving historical states, simulating "what-if" scenarios by editing and analyzing versions within temporary sandboxes, and comparing or merging divergent versions. In response to this broad need, many systems have been developed, including GeoGig [27], Kart [33], and others [35, 47, 62].

Nevertheless, the performance of these systems remains suboptimal due to a lack of support for parallelism; yet, enabling parallelism is highly challenging due to the intricacies of version access. One traditional approach to versioning is the spatio-temporal indexes [35, 47, 62], which casts temporal evolution as an additional time dimension. This solution, however, only supports updates to the tip of the timeline, thereby restricting it to linear versioning. Arbitrary version evolution is considerably more complex. Under Git-style version-control semantics (see Fig. 1), versions form a DAG, making it challenging to efficiently manage, modify, and query disparate versions, let alone in parallel. To our knowledge, no existing solution incorporates effective parallel updates, either for traditional spatio-temporal indices [35, 47, 62] or for Git-style frameworks such as GeoGig [27] and Kart [33]. As single-core CPU performance has largely plateaued since 2005, sequential solutions are increasingly unlikely to scale with the rapid growth of geospatial data. The closest parallel mechanism we are aware of, Multi-Version Concurrency Control (MVCC) [22, 36, 38, 40, 45, 48, 53, 58, 62, 65], supports concurrent updates but serves a fundamentally different purpose: it creates new versions atop the latest one upon updates, so that unfinished queries see a consistent version, rather than supporting expressive cross-version analysis or branching updates.

In this paper, we address this gap by introducing a **highly efficient, parallel, in-memory solution for managing spatial data with Git-style versioning**. Our system, SILVA[1], facilitates parallel updates and high-performance queries across arbitrary historical versions. By leveraging effective *parallelism*, SILVA **outperforms all existing baselines by *up to two orders of magnitude* for most update operations on a 96-core machine**. Furthermore, it demonstrates significant speedups across various query types supported by state-of-the-art systems.

Our work is motivated by recent designs of in-memory parallel functional (immutable) binary search trees (BSTs) [20, 21, 59–61], which have demonstrated effectiveness on supporting snapshot isolation on 1D (key-value) data. These tree-based data structures operate similarly to their mutable counterparts, but ensure immutability by a *path-copying* (or copy-on-write) strategy: instead of modifying the structure in-place, an update operation copies all nodes along the affected paths (an example of our design is presented in Fig. 3).

[1] Silva is the Latin word for "forest," symbolizing a collection of version trees.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGSPATIAL '26, Riverside, CA, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/06

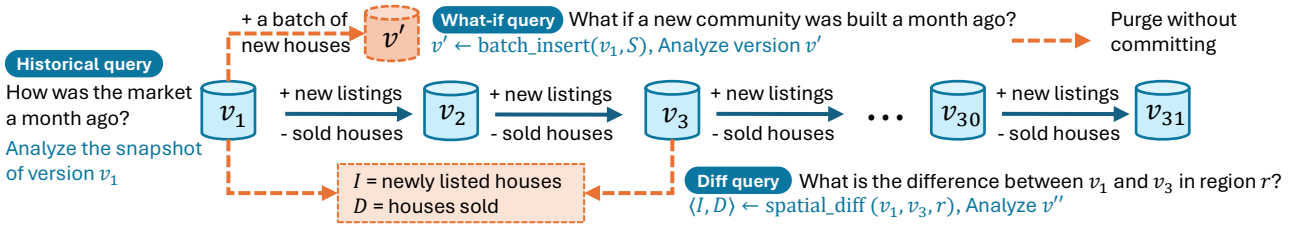

Figure 1: An illustration of Git-style version access on spatial data using an example on a real estate database. The dataset itself continuously evolves with a primary timeline of versions, updated by batch_insert and batch_delete for added and removed objects. Queries may need to investigate a historical version, create an experimental branch/sandbox (what-if query), or compare the diff between two versions.

Specifically, given an update batch representing a new version, we employ a parallel algorithm that performs insertions and deletions by generating new nodes for all impacted paths, thereby leaving the preceding version entirely intact. Since the root is a common ancestor to all nodes, each update inherently produces a new root pointer, which serves as a unique entry point for accessing that specific version's snapshot. Consequently, queries on any version are as efficient as those performed on a single-version index. Since each version can be accessed in isolation using its root pointer, branching and cross-version operations can be easily expressed, making them suitable for Git-style version control. Despite these advantages and parallel algorithms proposed for the 1D functional trees, we are unaware of any work that extends the parallel functional trees to multi-dimensional spatial data, especially given the distinct partitioning and balancing logic required for spatial data.

To support functional spatial trees, our first attempt is to embed multi-dimensional data into 1D via space-filling curves (specifically the Morton- or Z-curve) and incorporate them into an existing parallel functional BST in the CPAM library [20] (PaC-tree). We refer to this solution as the PaC-Z tree. To adapt the data structure for spatial queries, we augment each subtree with its bounding box. Our experiments show that this simple idea already outperforms the best baseline, the MV3R-tree [62] (the state-of-the-art multiversion spatio-temporal index), while being more general by supporting arbitrary branching. To achieve higher efficiency, we observe that the embedding itself introduces overhead from multiple aspects, including computing the space-filling codes and balancing the BST via rotations, which facilitate 1D queries but not spatial ones; indeed, the bounding boxes can largely overlap in a PaC-Z tree, hence reducing the efficiency of spatial queries. To address these issues, our second and main approach is to directly design parallel algorithms with path-copying to the quad-tree, which we refer to as Multi-Version Quadtree (MVQ-tree for short). The key insight here is that the quadtree is a history-independent data structure, with subtrees partitioned by spatial medians regardless of the point distributions. As such, MVQ-trees are faster than PaC-Z trees on updates due to the simpler logic of their update algorithms, and on cross-version queries due to the fully aligned spatial partitions and subspaces.

In summary, SILVA encompasses the design, analysis, and implementation of two functional trees: PaC-Z trees and MVQ-trees. Our algorithms are efficient with provable work-span guarantees, with theoretical analysis shown in Sec. 6. SILVA enables preserving and querying any historical version, arbitrary branching from any version, and the comparison or merging of two independent versions, as well as reclaiming memory from unused parts of the indices. SILVA also supports a wide range of spatial queries, such as spatial range count and report, k-nearest neighbors, and spatial join. Accessing a snapshot takes only $O(1)$ time to retrieve the root pointer, and standard spatial queries—such as k-nearest neighbor (KNN) or range queries—can be performed on a snapshot with the same efficiency as in a single-version index. In practice, we conducted extensive large-scale scalability experiments against state-of-the-art multi-version spatial indices. The results demonstrate that our solutions outperform existing baselines by up to two orders of magnitude in processing time while requiring only one-third of the memory footprint. To summarize, the primary contributions of this paper are as follows:

(1) We formalize the problem of spatial multi-version data processing and define the set of index operations for this problem.
(2) We propose two spatial indexes, PaC-Z tree and MVQ-tree. Both of them support parallel algorithms with theoretical guarantees and flexible Git-style version maintenance.
(3) We develop PaC-Z tree and MVQ-tree in the library SILVA.
(4) We run experimental evaluations against state-of-the-art (SOTA) spatial indexes. Experiments show that SILVA significantly outperforms SOTA solutions due to better parallelism.

We make our code **publicly available at [55]**.

2 Background and Preliminaries

Parallel Computation Model. To design and analyze parallel algorithms, we use the classic *binary fork-join* paradigm [3, 13, 14] with work-span model. In this model, multiple (software) threads share memory. Each thread is a sequential RAM augmented with a *fork* instruction, which spawns two parallel child threads. The parent thread resumes execution when both children finish. A computation can be viewed as a directed acyclic graph (DAG) with vertices representing computation and edges representing parent-child relationships. The *work* (W) of a parallel algorithm is the total number of operations (aka. the time complexity in serial execution). The *span* (or *depth*) (D) is the length of the longest dependence chain among operations. Using a randomized work-stealing scheduler, a parallel execution with work W and span D can be executed in $W/P + O(D)$ time with probability at least $1 - W^{-c}$, for any constant $c > 0$ using P processors [13, 14, 28].

Preliminaries for Spatial Data. We describe our algorithms based on spatial data as 2D points in Euclidean space, where each point is identified by a unique ID and a location (x, y). Our techniques

can also be extended to shapes in low dimensions, as discussed in Sec. 11. Many classical spatial indexes exist, including *quadtrees* [23], *k*-d trees [6], and *R-trees* [5, 29, 37, 47, 54, 62, 63]. They generally work by grouping nearby points in a hierarchical structure to allow quick top-down searches.

One common indexing method is to use a *space-filling curve* (SFC). An example is the Hilbert curve, or the Z-curve [46] as shown in Fig. Fig. 2. It encodes each point as an integer along the Z-shaped curve, which effectively linearize 2D data into 1D with a total ordering. SFCs are widely used in spatial indexes [25, 50–52, 57]. Our solutions PaC-Z tree and MVQ-tree also use Z-curve.

Functional Data Structures (FDSs). Functional data structures are *immutable* data structures in which data cannot be modified in place. Instead, updates produce a new version of the structure, preserving the original version. To avoid duplicating the entire structure, a typical approach for tree-based FDSs is *path-copying* (or *copy-on-write*), where only nodes along the update path are copied, allowing unmodified nodes to be shared between versions (see examples of our designs in Fig. 2 and Fig. 3).

FDSs naturally support version access by maintaining previous states as immutable snapshots, enabling Git-style branching. For tree-based FDS, each version can be uniquely represented by the root pointer. This design ensures *atomicity* for complex updates as a transaction: if an update contains multiple operations (e.g., a batch of insertion/deletions), they become visible altogether when the root is available. It also ensures query *consistency*: once a query reads the root pointer to acquire a version, it operates on a consistent snapshot of the data, unaffected by concurrent updates.

Path-copying has been adopted by several database systems [1, 7, 24, 40, 60] for hybrid transactional and analytical processing (HTAP) workloads. Some of them are also parallel [60]. Notably, PaC-tree [20] is a state-of-the-art parallel FDS with space optimization, which our PaC-Z tree is built upon. However, these parallel data structures all focus on the general 1D key-value store. For industrial spatial indexes, the GeoGig system [27] also uses path-copying for Git-style version managements, yet its updates are sequential. To the best of our knowledge, our work is the first to introduce parallel FDS to version-aware spatial indexes.

3 Problem Definition

We first define the functions of interest for the spatial index with Git-style version control. To capture parallelism, the interface takes updates in batches and processes them in parallel; a single update is simply the special case of a batch of size one.

To track the evolving status of a spatial dataset, we use *version* to refer to a certain state of spatial index on P, and each version is represented and accessed by a *handle* (denoted by h). In SILVA, where the index is a path-copying tree structure, the handle is just the root pointer of the tree.

The Interface of a Multi-version Spatial Index. We now define the interface for a multi-version spatial index. A multi-version spatial index should provide the *same* set of operations as a regular spatial index (e.g., construction, updates, and spatial queries), as well as version-control operations, which are useful in supporting and tracking Git-style workload, such as branching, merging, and auditing. For a multi-version spatial index, a read-only query on any version should be *as easy as* querying a regular spatial index, e.g., RangeCount, RangeReport, Knn, and SpatialJoin, so we omit their detailed definition for brevity.

Build(P): Given a spatial dataset P, initialize a new index and return the initial version handle.

BatchInsert(h, I): Given a version handle h and a point set I, insert points in I into h and return the new version handle.

BatchDelete(h, D): Given a version handle h and a point set D, delete points in D from h and return the new version handle.

Commit(h, I, D): Given a version handle h, an insert point set I and a delete point set D. Return the new version handle after deleting points in D from h and inserting points in I into h.

SpatialDiff(h_1, h_2, r): Given two version handles h_1, h_2, and an *axis-aligned rectangle region* r. Return a pair of point sets, which represent the minimal change of points (insert or delete) within region r that transform from h_1 to h_2.

Merge(h_1, h_2, S): Given two version handles h_1, h_2, and a conflict resolving strategy S. Return the new version handle by combing points in h_1 and $h2$. Any conflicts are resolved by using strategy S.

Purge(h): Given an unused version handle h, delete and *free* the memory space for h, without affecting other existing versions.

While this work mainly focuses on main-memory indexes, SILVA supports load/store disk operations for durability which are described in Sec. 12.

Notice that it is straightforward to implement these operations by making a full copy of the index with each operation. However, this would be extremely inefficient due to the redundant storage of non-modified parts. The challenge is to implement these operations while reducing redundancy between index versions as shown further in the next sections.

4 The Design of the PaC-Z Tree

To leverage parallel multi-versioning ideas on 1D data, we build upon PaC-tree [20], a SOTA 1D parallel functional tree, by integrating it with a space-filling curve (SFC), specifically the Z-curve [46]. Below, we first overview the existing design of PaC-tree and then outline the high-level idea of our design of PaC-Z tree. Due to space limitations, we defer detailed algorithms to the appendix.

The PaC-tree. The PaC-tree [20] is a binary search tree with leaf wrapping. Structurally, each internal node maintains an entry and two child pointers, while each leaf encapsulates an array of $\Theta(B)$ entries (with B typically set to 32). Tree algorithms achieve parallelism through a divide-and-conquer concat-based algorithmic framework[9, 12, 61]. All update operations in PaC-tree support path-copying for multi-versioning and snapshot isolation. Inspired by these capabilities, our PaC-Z tree tree uses PaC-tree as the base design and extends it for spatial data processing.

Our PaC-Z tree. To leverage the 1D parallel update algorithms inherent in PaC-tree, PaC-Z tree maps each spatial data point to its corresponding Z-value, and use the Z-value as the search key for a PaC-tree. To facilitate spatial search pruning, each tree node is augmented with a bounding box covering all records in its subtree. These bounding boxes are dynamically maintained in constant time during structural changes, such as node creations or rotations. Fig. 2 illustrates an insertion process in PaC-Z tree.

The core design philosophy of PaC-Z tree is to maximize the utility of proven 1D parallel versioning algorithms. This strategy yields a simple yet highly performant spatial index with minimal implementation overhead. Crucially, it enables the direct application of PaC-tree's 1D algorithms with inherent support of parallelism and path-copying-based versioning while preserving the original theoretical bounds (see Sec. 6). For instance, tree construction, batch insertion, and deletion algorithms are reused directly, while the set-union algorithm from PaC-tree naturally facilitates our version merging logic. Built upon that, we develop algorithms for spatial-aware query algorithms, such as k nearest neighbor (k-nn) queries, range queries, and spatial-diff.

Starting with queries applicable to any single version, we apply standard spatial pruning via bounding boxes. We can use bounding boxes to determine whether a subtree is irrelevant to a spatial query—e.g., when its box is out of the query range in a range report query, or the box is further than the current k-th nearest candidate in a k-nn query—and skip the entire subtree if so. Range count queries can be further accelerated when the subtree is entirely within the query range, and the cardinality of the subtree (which is stored within each tree node) can be directly added to the result.

Beyond the standard spatial queries applicable to a single version, we also introduce version-aware query algorithms, which offer more sophisticated querying capabilities. For spatial-diff queries, the high-level idea is to traverse two trees T_1 and T_2 simultaneously, and detect the unchanged sub-regions by comparing tree nodes. If the two trees shared the same subtree root, then there are no changes to report (i.e., $T_1 = T_2$). Otherwise, we first use the `split` function in PaC-tree, which splits one tree (typically the larger one) by the root key r of the other tree. Without loss of generality, assume we split T_2 by T_1. Assume T_1 has left (right) subtree L_1 (R_1) with root key k_1, then we will use the `split` function to split T_2, obtaining two subtrees: $T_2^{<k_1}$ and $T_2^{>k_1}$, which contains all entries in T_2 that are less/greater than k_1, respectively. Then, we recursively process the subtree pairs: $\langle L_1, T_2^{<k_1}\rangle$ and $\langle R_1, T_2^{>k_1}\rangle$ to collect spatial diff results. This approach is similar to the one in our spatial-invariance aware index MVQ-tree, described in Sec. 5.

The PaC-Z tree approach successfully provides the full versioning semantics outlined in Sec. 3. As our experiments demonstrate, it also achieves substantial performance gains over baselines like GeoGig and MV3R-tree. Nevertheless, we observed that its reliance on SFC embedding introduces inherent inefficiency. For updates, performance is hindered by the tree rotations required to maintain a strict 1D order, which is unnecessary for 2D data. For queries, SFC leads to overlapping bounding boxes among sibling subtrees. This poor spatial locality severely degrades the effectiveness of search pruning, highlighting the potential for further optimization. Motivated by this, we provide our main design MVQ-tree, which directly applies path-copying on the spatial index quadtree.

5 The Design of the MVQ-Tree

To overcome the rotation overhead and spatial misalignment of PaC-Z tree, we propose our main solution MVQ-tree, a path-copying quadtree-based structure. The quadtree inherently enforces strict spatial alignment, and also eliminates the need for rotations, thereby ensuring highly efficient updates. Motivated by these advantages, our goal is to design a parallel quadtree with version access.

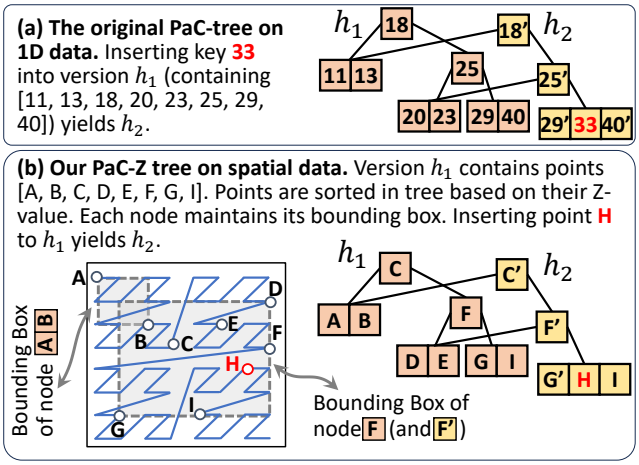

Figure 2: An illustration of the original PaC-tree [20] and our PaC-Z tree. Both subfigures show how path-copying is applied to the tree with an insertion example, where unmodified subtrees are shared. By embedding spatial data to 1D using Z-curve, PaC-Z tree effectively leverages existing parallel path-copying algorithms in PaC-tree.

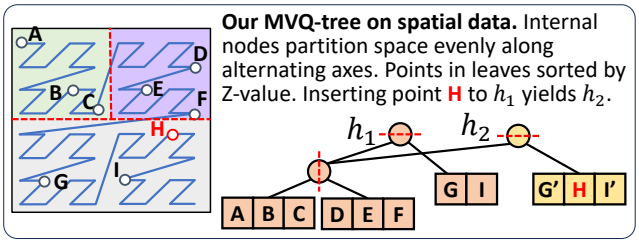

Figure 3: MVQ-tree Example. MVQ-tree directly applies path-copying to classic quadtrees. Unmodified regions (e.g., the upper part in the figure) are shared across versions.

While there exist parallel quadtree structures (e.g., the zd-tree [11] and P-orth tree [43]), they do not support versioning. To bridge this gap, we build MVQ-tree upon the parallel update mechanisms of the zd-tree [11] and introduce new algorithms to enable comprehensive version access, encompassing both updates and version-specific queries. Below, we first provide an overview of the design of MVQ-tree, followed by details of the algorithms for each operation.

Overview of MVQ-Tree. Following the Zd-tree [11], MVQ-tree also employs a binary space partitioning strategy: internal nodes partition the space using the spatial median along one axis, alternating between x- and y-axes at each level. Each internal node stores its subtree size (for faster range-count queries) and two child pointers. Note that bounding boxes are *not* stored in interior nodes since they can be derived on-the-fly during traversal. Points reside in the *leaf* nodes, each containing an array of $\Theta(B)$ (typically 32) points ordered by Z-values. Crucially, all partition boundaries are deterministic, making the structure well-suited for multi-version access: unmodified regions can be shared across versions, improving both storage efficiency and query performance.

Operations. We now present MVQ-tree algorithms. Throughout the paper, Z-values are assumed to be b^*-bit (typically 64) integers.

1. Build. The Build operation, illustrated in Alg. 1, takes a list of points P as input, and builds a Zd-tree over it. The algorithm

Algorithm 1: BUILD

Input: A point set P
Output: The initial version handle of P
Parameter: B: leaf capacity.
b^*: maximum Z-value binary representation length.

```
1  Function Build(P)
2      if P = ∅ then return nullptr
3      Calculate Z-values for points in P and sort them by Z-values in parallel
4      h ← BuildNode(P, 0, |P|, b*)                    // h is the version handle
5      return h
6  Function BuildNode(P, l, r, b)
7      if b = 0 or r − l ≤ B then
8          x ← new leaf node stores points {P_l, ..., P_r}.
9          return x
10     mid ← the minimum i ∈ [l, r) with the b-th bit in P_i as 1
11     In Parallel:
12         L ← BuildNode(P, l, mid, b-1)                // Build left subtree
13         R ← BuildNode(P, mid, r, b-1)                // Build right subtree
14     Let x be a new interior node with L and R as children
15     Update subtree size for x
16     return x
```

begins by computing and sorting the Z-values for all points in parallel, then recursively builds the tree via a helper function BUILDNODE(P, l, r, b), which operates on the l-th to the r-th points in P using the last b bits in their Z-values. At the top level $b = b^*$.

In BUILDNODE, if the input fits in one node (line 9), a leaf is simply returned; otherwise, it partitions $P[l..r]$ by the b-th Z-value bit (0 0 to the left, 1 to the right), finding the split position *mid* via binary search (line 10). Both subtrees are built recursively in parallel over $P[l..mid]$ and $P[mid..r]$, using the lower $b-1$ bits. The resulting internal node x (line 14) stores the subtree size as its augmented value and is returned as the version handle.

2. BATCHINSERT and BATCHDELETE. MVQ-tree processes updates in *batches* as described in Sec. 3. Due to space limit, we only detail BATCHINSERT here. The BATCHDELETE algorithm can be implemented similarly, and we present its details in the appendix.

The function BATCHINSERT(h, I) inserts a set of points I to a version with handle h, and returns a new version handle containing all points in h and I, keeping the original version h intact. We present the pseudocode in Alg. 2. If I is empty, h remains unchanged. Otherwise, we first calculate Z-values for points in I and sort them by Z-values in parallel. Finally, a helper function BATCHINSERTNODE(h, I, l, r, b) is called to process all points in the range $[l, r)$ in the insertion batch I, focusing on the last b bits of their Z-values, and insert them to the subtree rooted at h.

In BATCHINSERTNODE, we first check whether h is `nullptr`. If so, we directly call BUILDNODE (Alg. 1) on I, and return the new subtree root on line 6. If h exists, we duplicate it into a new node x to accommodate the incoming points, leaving the original version at h intact. Note that x shares unmodified child pointers with h. If x is a leaf node (line 8), we first determine whether it has sufficient capacity for the incoming points. If so, we merge them on line 10 and return the modified leaf node x. Otherwise, we consolidate all the points into a new sorted list I', and then build a new subtree, returning the new handle on line 14. Alternatively, if x is an interior node, we partition the insertion batch I by the b-th bit of the Z-value. By definition, those with the b-th bit as 0 are routed to the left child, while those with 1 go to the right. Therefore, we binary

Algorithm 2: BATCHINSERT

Input: A version handler h, a point set I to be inserted
Output: New version handle after insertion
Parameter: B: leaf capacity.
b^*: maximum Z-value binary representation length.

```
1  Function BatchInsert(h, I)                          // BatchInsert entry
2      if I = ∅ then return h
3      Calculate Z-values for points in I and sort them by Z-values in parallel
4      return BatchInsertNode(h, I, 0, |I|, b*)
5  Function BatchInsertNode(h, I, l, r, b)
6      if h = nullptr then return BuildNode (I, l, r, b)
7      x ← new tree node by copying h
8      if x is leaf then
9          if b = 0 or |x.points| + r ≤ B + l then
10             merge {I_l, ..., I_r} into x.points      // modify copied leaf
11             return x
12         else
13             I' ← x.points ∪ {I_l, ..., I_r}
14             return BuildNode (I', 0, |I'|, b)
15     mid ← the minimum i ∈ [l, r) with the b-th bit in I_i as 1
16     In Parallel:
17         if l < mid then x.lc ← BatchInsertNode(h.lc, I, l, mid, b-1)
18         if mid < r then x.rc ← BatchInsertNode(h.rc, I, mid, r, b-1)
19     Update subtree size for x
20     return x
```

search the boundary *mid* on line 15. Finally, we process the two children recursively in parallel, and update the size of the current interior node afterwards in line 19.

3. PURGE. MVQ-tree supports purging an existing version without affecting other versions. Following standard practice in functional data structures [20, 21, 61], we maintain a reference counter for each node to track the number of versions referencing it. To purge version h, PURGE recursively decrements these counters starting from the root. Nodes with a zero counter are unreferenced by any live version and are safely reclaimed.

4. COMMIT. COMMIT creates a new version from a conflict-free batch of mixed insertions and deletions. It applies BATCHDELETE and BATCHINSERT in turn, and then uses PURGE to discard the intermediate version created by BATCHDELETE.

5. SPATIALDIFF. Given two version handles h_1 and h_2, along with a rectangle r, SPATIALDIFF (Alg. 3) returns a point set pair $\langle I, D\rangle$, which signifies the minimal point insertions (I) and deletions (D) need to transform h_1 into h_2 within region r. For better clarity, we denote the pair as *spatial diff results*.

As shown in Fig. 4, the algorithm processes the spatial diff by advancing two pointers simultaneously and recursively comparing the subtrees they reference. SPATIALDIFF first computes the bounding box in line 5. Since MVQ-tree partitions by spatial median, the bounding boxes can be calculated along the tree path. Furthermore, the deterministic nature of quadtree partitioning ensures strictly synchronized pointer trajectories and aligned bounding boxes. Crucially, if a subtree region remains unmodified between h_1 and h_2, the subtree root is a shared node between the two versions. This allows for a quick equality check to bypass exhaustive element-wise comparisons. These properties significantly improve efficiency by enabling early pruning, as further detailed below. Based on comparing the two pointers, we describe five cases below:

Case I: identical nodes or outside query region: If the two pointers

Algorithm 3: SpatialDiff

```
Input: Two version handlers h1, h2, and a query rectangle r
Output: The insertion, deletion point sets can transform h1 to h2
Parameter: b*: maximum Z-value binary representation length.
1  Function SpatialDiff(h1, h2, r)
2      ⟨I, D⟩ ← SpatialDiffNode(h1, h2, r, b*)
3      return ⟨I, D⟩
4  Function SpatialDiffNode(h1, h2, r, b)
5      box ← current tree node bounding box
6      I ← ∅, D ← ∅
7      if h1 = h2 or box∩r = ∅ then return ⟨∅, ∅⟩  // Case I: identical nodes
8      else if h1 = nullptr and h2 ≠ nullptr then  // Case II: one nullptr
9          I ← range report results of h2 in query region r
10     else if h1 ≠ nullptr and h2 = nullptr then
11         D ← range report results of h1 in query region r
12     else if h1 is leaf and h2 is leaf then          // Case III: two leaves
13         ⟨I, D⟩ ← spatial diff results for h1.points and h2.points in r.
14     else if h1 is leaf and h2 is not leaf then // Case IV: one leaf, one interior
15         ⟨I, D⟩ ← spatial diff results for h1.points and h2 in r
16     else if h1 is not leaf and h2 is leaf then
17         ⟨I, D⟩ ← spatial diff results for h1 and h2.points in r
18     else
19         In Parallel:                                // Case V: two interiors
20             ⟨I_L, D_L⟩ ← SpatialDiff(x.lc, y.lc, r, b-1)
21             ⟨I_R, D_R⟩ ← SpatialDiff(x.rc, y.rc, r, b-1)
22         ⟨I, D⟩ ← ⟨I_L ∪ I_R, D_L ∪ D_R⟩
23     return ⟨I, D⟩
```

reference the same node (the region is unchanged) or the bounding box does not intersect the query region r, no differences are reported in line 7. Fig. 4 shows this case by moving from root along path ①.

Case II: one `nullptr`: If one pointer is `nullptr`, we have either inserted points to a previously empty region, or deleted all points from an existing region. Therefore, we report the other (non-null) node's points as insertions or deletions in line 8 and line 10.

Case III: two leaves: If the two pointers reference difference leaf nodes, SpatialDiff computes the inserted/deleted points for the points in two leaves by scanning in line 12. Fig. 4 demonstrates this case by following the path ② → ④. D and D' are two leaf nodes, so we scan them to obtain the inserted/deleted points. Since all points are sorted, we can achieve this by linear scanning.

Case IV: a leaf and an interior node: The next case is when two pointers refer to a leaf node and an interior node respectively. Without loss of generality, assume the pointer in h_1 is a leaf and that in h_2 is an interior node. We then continue to traverse down from h_2, and compare the points in its subtree with the leaf node h_1 (lines 14-17). As we traverse down h_2, we (conceptually) split the set of points in h_1 to align with the subtrees in h_2. Note that such a split does not make structural changes to the node, but just identifies the boundary in the list of points in the leaf. In example Fig. 4, when the two pointers traverse along tree path ② → ③. C is a leaf node in version h_1 and C' is an interior node in version h_2. We continue traversing the interior node side C', and logically split the leaf node side C (the dotted rectangles), so they can be compared with subtrees in C' respectively. Finally, we reach the leaf node under C' and can obtain the results.

Case V: two interior nodes: SpatialDiff handles the two interior nodes case in line 19 by recursively calling SpatialDiffNode for the child pointers of the two interior nodes. Since the two subtrees are aligned, we only need to compare the children in the same direction, i.e., left-to-left and right-to-right. Fig. 4 shows this case by following path ② → ③ and ② → ④.

Finally, the spatial diff results are merged and returned in line 22.

Algorithm 4: Merge

```
Input: Two version handles h1, h2, and conflict resolving strategy S.
Output: The handler of new merged version.
Parameter: r_g: The global region (entire space).
1  Function Merge(h1, h2, S)                          // Merge entry
2      h_b ← common base version handle of h1 and h2
3      In Parallel:
4          ⟨I1, D1⟩ ← SpatialDiff(h_b, h1, r_g)  // get changes from h_b to h1
5          ⟨I2, D2⟩ ← SpatialDiff(h_b, h2, r_g)  // get changes from h_b to h2
6      ⟨I, D⟩ ← no conflict updates by linear scan I1, I2, D1, D2
7      ⟨I', D'⟩ ← conflict updates by linear scan I1, I2, D1, D2
8      h_t ← h_b
9      if strategy S is provided then                  // solve conflict
10         h_t ← Commit I', D' to h_t according to S
11     h ← Commit(h_t, I, D)                           // commit not conflict points
12     Purge (h_t)
13     return h
```

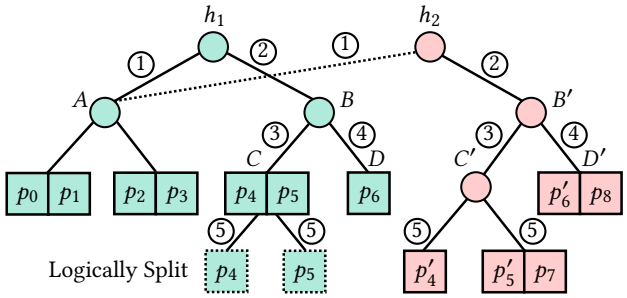

Figure 4: MVQ-tree Spatial Diff Example. h_1 and h_2 are two version handles. The numbers on the tree edges are used to describe tree paths.

6. Merge. Given two version handlers, h_1 and h_2, and a conflict-solving strategy S, the Merge function returns a new version handle containing all points in h_1 and h_2. The spatial merge operation is implemented based on SpatialDiff and Commit. Firstly, we calculate the common base version h_b for h_1 and h_2 in line 2. The base version can be achieved by tracking back along the version history. e.g., when versions are managed in tree-like style, we can get h_b by their *least common ancestor*.

Then we call SpatialDiff to compute the diff of the base version to both h_1 and h_2 in line 4 and line 5, respectively. After that, we merge the modification sets together in line 6 to obtain the non-conflict and conflict modifications, respectively. Since all points are sorted by Z-values, this procedure can be achieved by linear scanning the point sets. If a conflict-solving strategy is provided, we will commit the conflict updates according to the given strategy and obtain an intermediate version in line 9. Finally, we call Commit to commit the non-conflict changes in line 11, purge the intermediate version, and return the new merged version handle. Note that there is no extra space usage except for the merged version.

6 Theoretical Analysis

This section presents the work and span analysis for version operations in SILVA. We omitted spatial queries and merge since they are highly data sensitive and the analysis on the worst case becomes trivial. Since all three indexes use *leaf wrapping* technique, we use

B (usually 32) to represent the leaf node capacity and assume it is a constant.

Theorem 6.1 (PaC-Z tree Work and Span). *Assuming constant leaf sizes, for a point set P with n points, a PaC-Z tree can be built on P in $O(n \log n)$ work and $O(\log n)$ span. An insertion, deletion, or commit involving $m \leq n$ updated points can be performed on T with $O(m \log n)$ work and $O(\log m \log n)$ span.*

Proof of Thm. 6.1. **Build.** The build procedure first compute Z-values for all points, sort them in parallel, and finally build a perfect balanced binary search tree use divide-and-conquer manner in $O(n \log n)$ work with $O(\log n)$ span [20].

Batch Insertion (Delete and Commit are similar). Batch insertion consists of three steps: 1) partition m points into subtrees until meet the base cases; 2) process base cases (append inserted points to existing leaf; dealing with leaf overflow); and 3) rebalance after insertion. Firstly, node copy for a tree node has $O(1)$ cost, since PaC-Z tree nodes have constant size. For the partition part, T has $O(\log n)$ levels, and in each level the partition procedure costs $O(m)$ work and $O(\log m)$ span. Therefore, the partition work in total is $O(m \log n)$ and the partition span is $O(\log m \log n)$. For the base case, we need to touch all updated points in the worst case. Therefore the work is $O(B + m) = O(m)$, and the span is $O(\log(B + m)) = O(\log m)$. For the rebalance part, PaC-Z tree uses concat primitive [61] for rebalancing. Each concat can combine two trees with size x and y in $O(\log \frac{x}{y})$ work and span ($x > y$). There are $O(m)$ leaf nodes need to be rebalanced, and thus $O(m)$ concat will be called. Each concat costs at most $O(\log \frac{n}{m})$ work and span (w.l.o.g. assume $n > m$). Therefore the rebalance work is $O(m \log \frac{n}{m})$, and the span is $O(\log \frac{n}{m})$.

Adding up the three parts, the insertion work is $O(m \log n + m + m \log \frac{n}{m}) = O(m \log n)$, and insertion span is $O(\log m \log n + \log m + \log \frac{n}{m}) = O(\log m \log n)$, bounded by partition. □

Theorem 6.2 (MVQ-tree Work and Span). *Assuming constant leaf sizes, for a point set P with n points, an MVQ-tree of height h can be built on P using $O(n \log n)$ work and $O(h \log n)$ span. An insertion, deletion, or commit involving $m \leq n$ updated points can be performed on T using $O(m(h + \log m))$ work and $O(h \log m + \log n)$ span.*

Proof of Thm. 6.2. **Build.** The first step in the build algorithm is to compute and sort by Z-values, and then build a perfectly balanced binary search tree. The part for MVQ-tree is the same as PaC-Z tree, with $O(n \log n)$ work and $O(\log n)$ span.

For the tree construction step, a binary search will be invoked for each interior node to partition the input points. A partition for an interior node receives n_i points needs $O(\log n_i)$ work and span, and the total work for a level is $O(\sum \log n_i) = O(n)$ (since binary search is faster than linear scan). Thus, there are h levels and the total work is hn. Hence, the total build work is $O(n \log n + hn) = O(n \log n)$ (since h is bounded by 64 and can be treated as $O(\log n)$ in our design).

For the span, the longest tree path has length h (the tree height), hence, the span along this path is $O(h \log n)$.

Batch Insertion (Deletion and Commit are similar.) First we need to sort m points, which requires $O(m \log m)$ work and $O(\log m)$ span as mentioned above.

Then, for each interior node to be modified, we need to make a copy, and copy each of them only requires $O(1)$ work and span. In addition, for each touched interior node, we also need to partition the m updated points by binary search. This part is similar to build, and thus we can obtain the partition procedure has $O(hm)$ work, and $O(h \log m)$ span.

Finally, we need to merge existing nodes with update points or build a new subtree. For the former case, since the leaf node has constant size, the merge procedure can be processed by a parallel merge algorithm with $O(m)$ work and $O(\log m)$ span. For the latter case, we will call build as a subroutine, therefore the work and span are $O(m \log m)$ and $O(h \log m)$, respectively.

Combine all parts together, we obtain the batch insertion work is: $O(hm + m \log m) = O(m(h + \log m))$, and span is $O(h \log m + \log m) = O(h \log m + \log n)$. □

7 Experiments

We conduct extensive experiments on both real-world [49] and synthetic datasets [26] to illustrate the efficiency of SILVA in maintaining spatial indexes with versioning. We will show that SILVA **outperforms existing multi-version spatial indexes significantly** in terms of build, batch insertion/deletion, spatial queries, and incurs a much **smaller memory footprint**. As a highlight, a case study on real-world data demonstrated that, both data structures in SILVA are **consistently faster** than the state-of-the-art spatio-temporal index MVR-tree [39], while only using 25.6–30.5% memory space. Specifically, compared to MVR-tree, SILVA achieves 38–74× speedup in tree construction and batch update, even sequentially, and **493–1511**× speedup when executed in parallel. For spatial queries, MVQ-tree is competitive or outperforms all tested baselines, which includes the well-recognized single-version R-tree in *Boost* [15]. SILVA also achieves high parallelism, exhibits linear scalability with respect to the number of processing cores. Below we will present more details about experimental setting and results.

7.1 Experimental Setup

Setup. All experiments were conducted on a 96-core machine with four-way Intel Xeon Gold 6252 CPUs and 1.5 TB of memory. All indexes are implemented in `C++` and compiled by `g++ 14.2.1` with `-O3` flag. We use parallel primitives from ParlayLib [10], and `numactl -i all` for parallel experiments, which interleaves memory across CPUs on NUMA architectures. Results are reported as the average of three test rounds after a warm-up run.

Baselines. We test SILVA against existing approaches: MVR-tree, MV3R-tree, and R-tree. All these baselines are sequential. MVR-tree and MV3R-tree are the state-of-the-art multi-version spatial indexes [62]. MV3R-tree maintains an extra 3D R-tree compared with MVR-tree to accelerate cross-version queries. We include MV3R-tree in our evaluation since it serves as a complete solution. We tested both MVR-tree and MV3R-tree as they are the SOTA multiversion spatial indexes. They can only modify the latest version, precluding Git-style operations (e.g., branching). We use MVR-tree implementation in *libspatialindex* [39]. Since its original code is not available, we implement MV3R-tree based on *libspatialindex*. boost-R is a well-recognized open-source single-version R-tree implementation in *Boost Geometry Library* [15]. We compare to

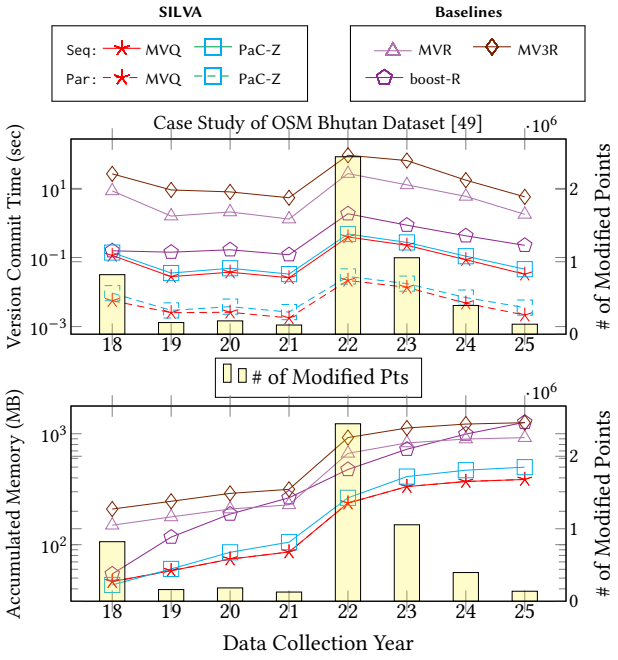

Figure 5: Real-world case study on *Version Commit Time* and *Memory Usage* (OSM Bhutan from 2018-2025), lower is better. *Solid* lines are measured *sequentially*. *Dashed* lines are measured in *parallel* (96-cores). Bars show the number of modified points in each year.

Table 1: Dataset Statistics

Dataset Type	Name	# of Points
Real-world [49]	Uzbekistan	10.4M
	Malaysia-Singapore-Brunei	27.2M
	Kazakhstan	29.3M
	Thailand	37.8M
	Nepal	67.7M
	Japan	95.1M
Synthetic	Uniform	0.1M to 1B
	Varden [26]	0.1M to 1B

boost-R with quadratic split to support our claim that SILVA adds versioning without query overhead over single-version indexes. Leaf capacities of all approaches are set to 32.

To evaluate the effectiveness of MVQ-tree SpatialDiff, we implement three baselines. The first two, namely MVQ-Post and boost-R-Post, represent the naive approach using single-version indexes. They call range report on the two versions (using Zd-tree [11] and R-tree, respectively), and compute the difference by post-processing. The third baseline, MVQ-IND, is designed to show the performance gains of MVQ-tree structural sharing. It employs the same spatial diff algorithm (as presented in Alg. 3), but operates on two independent MVQ-trees without any shared nodes.

Dataset. We run experiments on both real-world and synthetic data. For real-world data, we use *Open Street Map* (OSM) [49]. For synthetic data, we generate using `Uniform` and skewed `Varden` [26] distributions with coordinate range $[0, 10^7]$. The statistics of used datasets are discussed at the beginning of each evaluation section, and a summary of them is shown in Tab. 1

7.2 End-to-end Performance with a Case Study

To evaluate the end-to-end performance with a real-world workload, we used the yearly commits from the *OSM Bhutan* [49] dataset. We started with the initial dataset from 2018 and simulated the history of the yearly changes until 2025. All changes in one year are applied as a batch commit with a mix of insertions, deletions, and updates. Note that there are sudden bursts of point modifications in year 2022 and 2023, which contain more modified points than existing points before 2018. We measured both time cost and memory usage for each Commit function call. The yellow bars show the number of modified points in each year. We also show the sequential running time of SILVA since the baselines are all sequential.

Time Cost. The upper part of Fig. 5 shows the commit time in seconds (log-scale) for each version. For all tested indexes, the version commit time is proportional to the batch size, as expected.

The sequential version of SILVA already consistently outperforms all baselines by up to 68×. boost-R is slightly faster than MV3R-tree and MVR-tree, but it does not support versions. When running in parallel (96 cores), MVQ-tree, PaC-Z tree achieve on average 986×, 690× speedup compared with MVR-tree, and on average 69×, 49× speedup compared with boost R-tree. In SILVA, MVQ-tree has the best overall performance since it does not need rebalance as mentioned in Sec. 5.

Memory Usage. The lower part of Fig. 5 shows the accumulated memory in MB (log-scale) to maintain all history versions. Boost R-tree has to deep copy the previous version, and thus its memory growing regardless of whether the number of modified points is large or small (e.g., from 2018 to 2019). Such high memory overhead makes it less competitive than others in supporting versions. For this reason, we exclude boost-R in some of the future experiments.

All other techniques are version-aware, with memory increasing proportionally to the batch size of modified points. MVR-tree and MV3R-tree consume more memory due to their large node sizes [39]. PaC-Z tree has the lowest memory usage in 2018, since it is perfectly balanced. MVQ-tree consistently achieves the smallest memory footprint in years beyond 2018. This is because MVQ-tree does not modify a tree node unless changes apply to its subtree. In contrast, PaC-Z tree needs to copy more tree nodes due to tree rebalance and spatial misalignment. As such, MVQ-tree is superior than tested methods in terms of memory footprint for version management.

Query Performance. Beyond achieving better time and space efficiency, SILVA does not sacrifice spatial query performance. For the sake of limited space, please refer to Sec. 7.7 for details.

SILVA v.s. GeoGig. We also tested the case study workload for SILVA and GeoGig, and the running time is shown in Tab. 2. .

7.3 Static Construction

We now provide an in-depth study of the performance of the construction time of an index, with the results given in Fig. 6. Since the baselines are sequential, we also report our sequential time for comparison.

Time Cost. As shown in Fig. 6 , even sequentially, SILVA consistently outperforms the SOTA multi-version spatial indexes (MVR-tree and MV3R-tree) by around two orders of magnitude. The good performance of SILVA comes from its bulk load operation, while

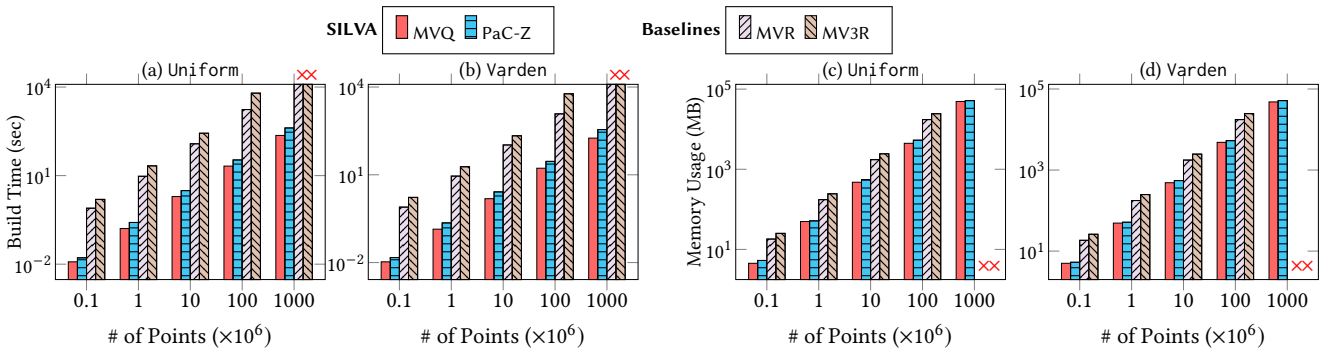

Figure 6: Build Time (Sequential) and Build Memory evaluation on synthetic dataset, lower the better. (a), (b) show the build time in seconds. (c), (d) show the Memory Usage in Megabytes. All axes are in log-scale. ✕ means timeout or not applicable.

Table 2: Case Study on GeoGig v.s. SILVA (time in seconds)

Year	Total Changes	GeoGig (Base)	GeoGig (QuadTree)	cpambb-seq	cpambb-par	mvzd-seq	mvzd-par
2018	Initial Import	1.7	23.8	0.139	0.009	0.115	0.006
2019	142,778	3.3	10.6	0.035	0.003	0.028	0.003
2020	146,387	3.4	9.1	0.049	0.004	0.037	0.003
2021	119,039	3.3	29.9	0.033	0.003	0.026	0.002
2022	2,231,190	3.5	73.1	0.478	0.028	0.393	0.022
2023	997,516	4.0	45.0	0.273	0.017	0.225	0.014
2024	347,683	3.6	103.8	0.109	0.007	0.086	0.005
2025	120,668	3.6	14.2	0.046	0.004	0.033	0.002

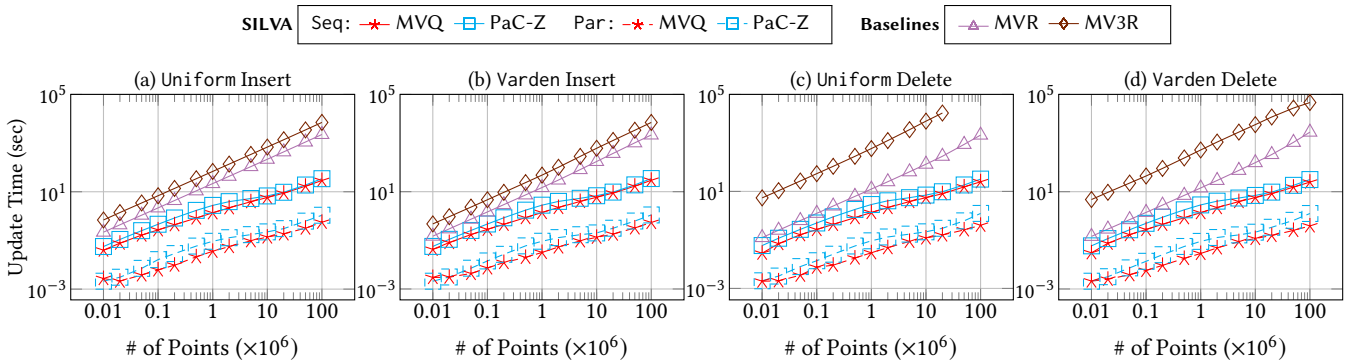

Figure 7: Batch insert/delete performance on initial version with 100M points synthetic datasets, lower the better. (a), (b) show insert time in seconds. (c), (d) show delete time in seconds. All axes are in *log* scale. Solid lines run sequentially, and Dashed lines run in parallel (96-cores).

MVR-tree and MV3R-tree can only be built incrementally. Consequently, MVR-tree and MV3R-tree timed out (> 3 hours) on the dataset with 10^9 points. In SILVA, MVQ-tree achieves consistently the best construction performance, since unlike PaC-Z tree, MVQ-tree avoids the overhead of calculating and storing extra information (such as MBR and Z-value).

Memory Usage. As shown in Fig. 6, MVQ-tree/PaC-Z tree use on average 72.5%/70.2% less memory than MVR-tree, respectively. This is because nodes in MVR-tree and MV3R-tree [39] are much larger: it stores start/end timestamps, low/high corners for points, augmented bounding box, and data pointers. It results in about four times larger tree node size compared with MVQ-tree.

In SILVA, PaC-Z tree uses more memory (although it is balanced) since it stores extra information such as Z-values and MBRs to locate points and accelerate spatial queries. In addition, despite MVQ-tree not achieving the best in single-version memory usage, it has the smallest accumulated memory usage when handling multiple versions, as we show in the case study Fig. 5.

7.4 Batch Insert/Delete, Commit, and Merge

BatchInsert/Delete. We start with a fixed index of 100M points and perform a batch insert/delete with batch sizes ranging from 10^4 to 10^8 (batches smaller than 10^4 take negligible time).

Fig. 7 shows the insertion and deletion times on the Varden dataset. Although SILVA performs comparably to the baselines for small updates, it becomes significantly faster when the batch size increases. This is because MVR-tree and MV3R-tree cannot take advantage of batch updates and must update points one at a time. In contrast, SILVA has designated batch update algorithms (see Sec. 4 and 5), leading to the increasing gap as the batch size becomes larger. When leveraging 96-core parallelism, MVQ-tree and PaC-Z tree are up to three orders of magnitude faster than MVR-tree, and up to 12,971× faster than MV3R-tree. In SILVA, MVQ-tree is

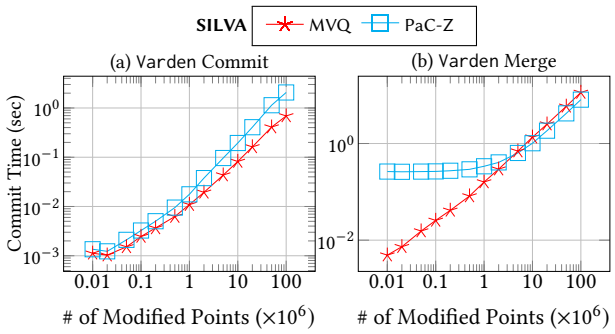

Figure 8: Commit and Merge time on Varden with 100M points, lower is better. Modification consists of 50% insertion and 50% deletion.

consistently the fastest, since it avoids tree-balancing overhead and benefits from more shared nodes.

Commit and Merge. As shown in Fig. 8 (a), we evaluate the Commit operation under a mixed workload (50% insertions, 50% deletions). Since Commit consists of BatchInsert/Delete followed by a cheap Purge operation, they have similar performance as batch updates. Note that MVQ-tree is the fatest in SILVA, and we didnot include other baselines since SILVA already proves its batch update efficiencies in the above discussion.

For the Merge operation, we reuse the Commit workload to create two versions (one for insertions, one for deletions). The Merge essentially consists of a diff and a commit operation. One interesting observation is that MVQ-tree is two orders of magnitude faster than PaC-Z tree for a small number of changes. This is due to the efficient spatial diff algorithm that advances two pointers simultaneously to quickly prune large portions of unchanged regions. In contrast, PaC-Z tree has a significant overhead in collecting all changes in the two versions, even with very small batch sizes. In practice, users tend to use the Merge operation when there are only a few changes, which is what the algorithm in MVQ-tree is optimized for. As the modification eventually affecting all points, MVQ-tree pruning improvement becomes negligible, and the performance of all methods converges. MVQ-tree takes slightly longer due to the overhead of spatial diff algorithm as detailed below.

7.5 SpatialDiff

To evaluate the SpatialDiff function, we first Commit a new version with the same workload as Sec. 7.4. Then, we generate random query regions and classify them based on the number of points they contain prior to the commit: $[0, 100)$ for *small* regions and $[100, 10{,}000)$ for *medium* regions. We exclude large regions since the cost will be the same as scanning the entire dataset. We also implement three SpatialDiff baselines (described in Sec. 7.1) to prove the effectiveness of our MVQ-tree design.

Fig. 10 shows that MVQ-tree is the clear winner for small modifications, i.e., less than 10%, and remains highly competitive for larger changes. This validates the effectiveness of our proposed spatial diff algorithm (Alg. 3). The comparison between MVQ-IND to MVQ-Post shows the effectiveness of the dual tree traversal algorithm even without node sharing. Interestingly, boost-R performs well for small regions where the range report queries are lightweight that are quick to compare. However, it becomes the slowest for medium regions since the query results become significantly larger. Finally, PaC-Z does not perform well on spatial diff queries, due to overlapping bounding boxes introduced by the Z-curve layout, which limits the effectiveness of pruning during range reporting.

Overall, MVQ-tree has the best performance of spatial diff queries, especially for detecting differences with moderate modifications.

7.6 Parallel Speedup

We now evaluate the self-relative speedup of SILVA indexes across varying core counts. We test a dataset of 10^8 points with 10% modification, consistent with the methodology in Sec. 7.4.

As shown in Fig. 9, all SILVA indexes achieve linear self-relative speedup in build time as the number of cores scales to 48. The MVQ-tree maintains consistently high speedup, attributed to its compact tree node design. Regarding the merge operation, Fig. 9 indicates that MVQ-tree exhibits slightly lower speedup at higher thread counts, which we attribute to the overhead of parallel memory allocation during traversal. In summary, all SILVA indexes achieve excellent parallel scalability.

7.7 Standard Spatial Queries

SILVA provides snapshot isolation: each version can be treated as a single-version spatial index. Therefore, existing single-version spatial queries can be seamlessly ported to SILVA without any additional overhead. We implemented *range report*, *range count*, *kNN*, and *spatial join* for SILVA to demonstrate its extensibility.

Fig. 11 (a) shows a scatter plot of *range report* times with respect to the query output size. Even though SILVA indexes provides access to all historical versions, their query performance is competitive with the highly optimized boost-R. This shows that SILVA does not have to trade off in performance to provide all its unique features.

Fig. 11 (b) presents similar results for the *range count* queries. We exclude MVR-tree and R-tree from this evaluation since they lack specialized range count algorithms and essentially use range report. In contrast, MVQ-tree and PaC-Z tree achieve 92× and 10× self-speedup respectively compared with range report due to subtree size augmentation. Note that MVQ-tree obtains significant self-speedup due to its perfect spatial alignment.

Fig. 11 (c) shows the results of the *kNN* queries while varying k from 1 to 100. MVQ-tree is comparable to, or even faster than the highly optimized boost-R. Both MVQ-tree and boost-R achieve at least an order of magnitude speedup over other techniques.

For MVQ-tree, we also implemented a spatial join (report) operation based on *synchronized traversal* [31], which finds all pairs of points across two versions that fall within a specified distance bound. We prepare two versions with 10^6 points (in Varden distribution), setting the distance bound to 50,000. Fig. 11 (d) plots the spatial join time as the input size increases. MVQ-tree can process a spatial join report in 2 seconds, which shows its extensibility in supporting existing spatial queries.

8 Real-world Data Evaluation

To verify the generalization of our results, we evaluated essential operations (including build, commit, merge) using up to 100M records from the OpenStreetMap (OSM) real-world dataset.

Fig. 12 (a) shows that SILVA maintains its performance advan-

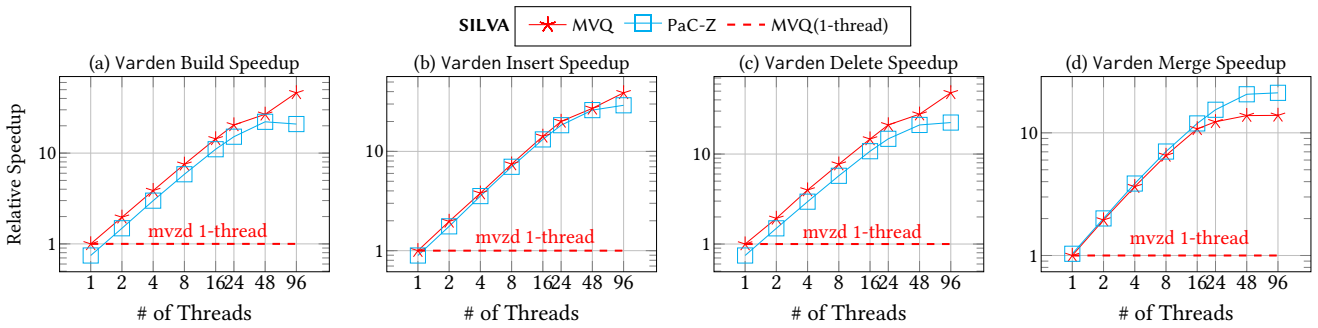

Figure 9: Parallel speedup w.r.t.# of threads on synthetic Varden datasets with 100M points, higher the better. The relative speedup is calculated w.r.t. MVQ tree 1-thread (shown in the dashed horizontal line) performance.

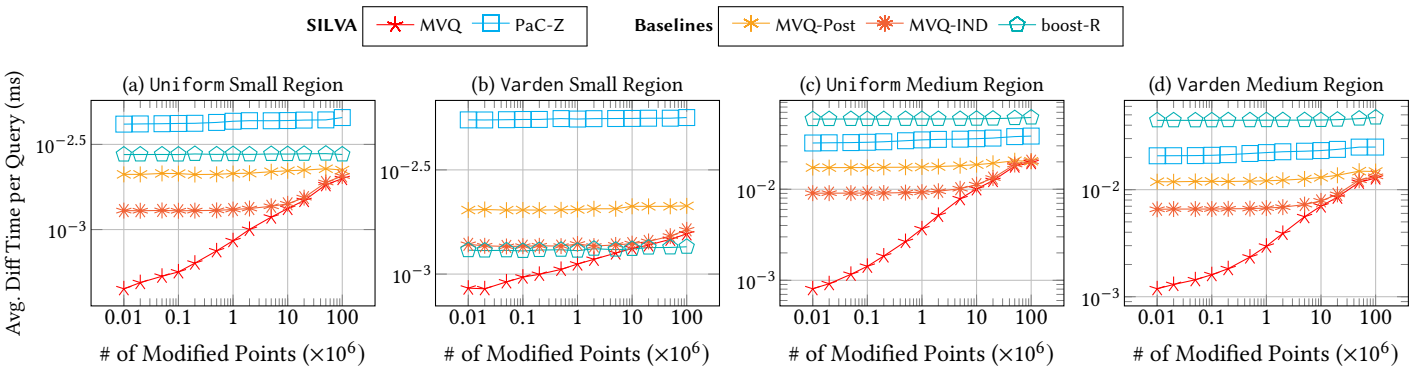

Figure 10: Spatial diff (sequential) on synthetic datasets with 100M points for small/medium regions, lower the better. Modified points consists of 50% insertion and 50% deletion. Each small/medium region contains $[0, 100)/[100, 10000)$ points before modification, respectively. All axes are in *log* scale.

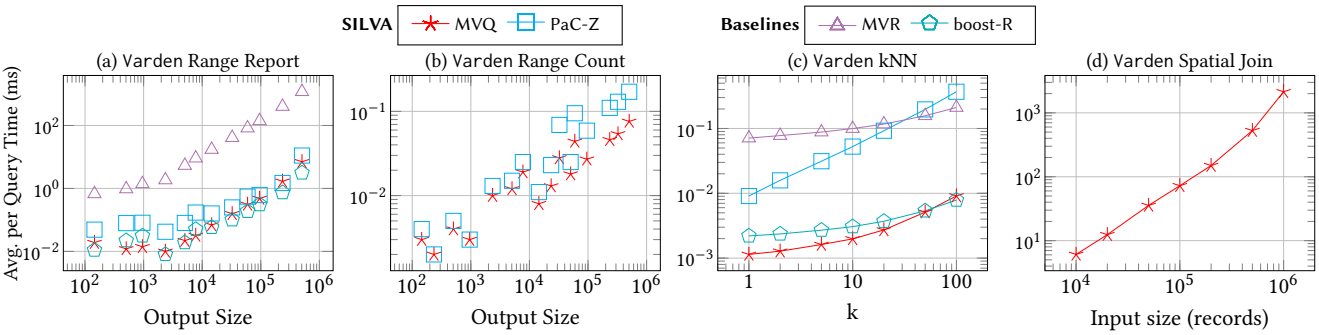

Figure 11: Single-version query (sequential) evaluation on synthetic datasets with 100M points, lower the better. *x*-axis is the parameter studied (i.e., # of points, output size, or k). All axes are in log scale. For (a), # of points is chosen before modifications. All queries are read-only and can be parallelized.

tage over MVR-tree, achieving up to two orders of magnitude speedup during index construction. Fig. 12 (b) presents the construction memory footprint. Due to its compact design, SILVA significantly reduces memory consumption compared to MVR-tree. Although MVQ-tree incurs a slightly larger initial footprint, as shown in Sec. 7.2, it conserves space during subsequent updates.

Fig. 12 (c) and (d) show the performance of commit and merge operations on the OSM Japan dataset (with about 100M points). Following our earlier setup, we select modification subsets of increasing sizes, splitting each into 50% insertions and 50% deletions. The results are consistent with our previous findings shown in Fig. 8.

To summarize, SILVA is equally efficient on real-world datasets as it is on synthetic data.

9 PaC-Z tree v.s. MVQ-tree

Design Philosophy: Both PaC-Z tree and MVQ-tree leverage functional data structures to support Git-style versioned access with leaf wrapping for spatial data. Specifically, PaC-Z tree is designed to maximize the utility of state-of-the-art 1D parallel libraries, which are proven to be theoretically sound and practically fast. Although PaC-Z tree already outperforms the test baselines significantly, its performance on spatial queries is unsatisfactory due to spatial misalignment. In contrast, MVQ-tree completely discards existing 1D parallel tree frameworks, and use an unbalanced but strictly spatially aligned quadtree design. Although MVQ-tree incurs some overhead due to its unbalanced structure, it achieves higher efficiency in spatial queries, particularly for version-based operations.

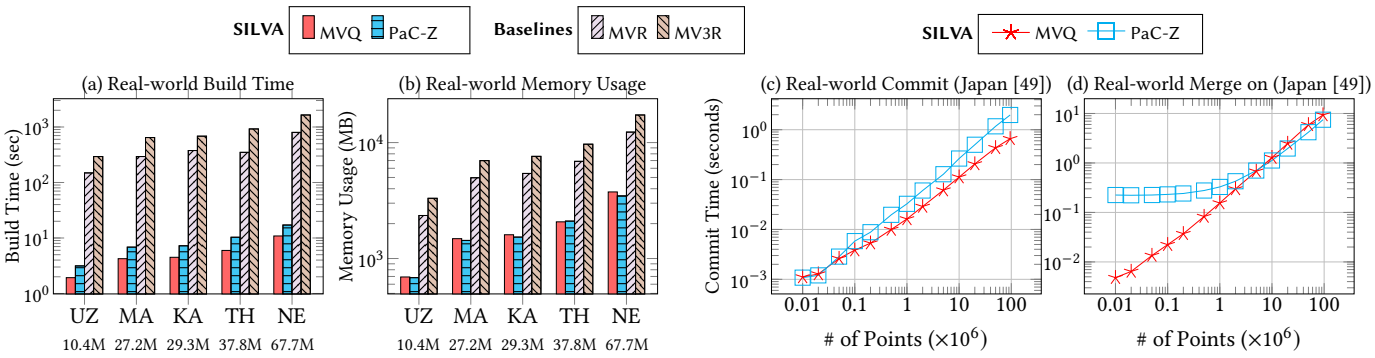

Figure 12: Real-world OSM data evaluation (in different regions), lower is better(log scale). UZ, MA, KA, TH, NE are all real-world datasets with different sizes. Please refer to [56] for detailed dataset statistics. The Japan dataset has roughly 100M points.

Operation Running Time: For most single-version operations, PaC-Z tree shows comparable performance compared to MVQ-tree. Although PaC-Z tree stores extra information, such as bounding boxes, it can still benefit from its balanced tree structure. However, for MERGE operation, PaC-Z tree needs to pay a significant range report overhead, which is not required in MVQ-tree. In addition, MVQ-tree has better spatial alignment, and thus it has better MERGE performance.

Memory Usage: For single-version management, PaC-Z tree has the lowest memory usage. This is expected, as PaC-Z tree is a perfectly balanced binary search tree. However, when managing multiple versions (as discussed in the case study Sec. 7.2), the frequent structural rebalancing in PaC-Z tree destroys the node-sharing opportunities across versions. In contrast, although MVQ-tree is not balanced, it preserves shared nodes in unmodified spatial regions, and thus lead to significantly lower memory footprint.

Spatial Query Time: Among all tested spatial queries, MVQ-tree achieves consistently better performance. The reason is that the bounding boxes in PaC-Z tree have many overlaps (as shown in Fig. 2), which may lead to extra subtree search. For MVQ-tree, its query search is guided by disjoint bounding boxes (see Fig. 3), and thus better spatial locality can be achieved.

10 Related Work

Single-version Spatial Indexes. Spatial indexes are fundamental to database systems, with traditional structures such as R-trees [5, 29], quadtrees [23], and k-d trees [6] designed for sequential processing. Recent efforts have introduced parallelism to these indexes, including bulk-loading and batch-update optimizations for R-trees [2, 32], Zd-trees [11], and others [43, 44, 52]. Despite better performance, all such approaches are limited to single-version operations. These indexes typically rely on in-place updates, which are incompatible with the Git-style versioning supported by SILVA.

Multi-Version Concurrency Control (MVCC). Modern in-memory databases increasingly adopt latch-free concurrency control, typically leveraging version chains or copy-on-write (CoW) mechanisms to enable high parallelism and avoid the contention of in-place updates [17, 34, 36, 40]. While these techniques effectively isolate concurrent transactions, their primary objective is to maintain consistency for the *latest* version. Once a transaction commits and old versions become obsolete, they are typically reclaimed by garbage collection. In contrast, our work requires the preservation of arbitrary historical versions for persistent access, auditing, and analytical querying. Unlike standard MVCC, which treats versioning for transaction isolation, SILVA treats versioned states as immutable and queryable entities, as the main goal of the system.

Non-spatial Versioning Approach. Several non-spatial systems explore versioning to support analytical workloads. Early concepts like hypothetical databases [64] use virtual indexes for what-if analysis, while the Bw-tree [38] manages versions by appending delta logs to tree nodes. More recently, systems such as DataHub [8], Decibel [42], and OrpheusDB [30] have introduced Git-style version control for relational tables. However, these systems do not natively support spatial indexing, rendering both single-version and cross-version spatial queries highly inefficient.

Multi-version Spatial Indexes. Several multi-version spatial indexes have been proposed in the literature. Kolovson and Stonebraker [35] manage versions by combining segment trees and R-trees. Historical R-trees [47] extended the path-copying technique to R-trees for multi-version spatial data. MV3R-tree [62] combines MVR-tree (an R-tree variant of the MVB-tree [4]) and a 3DR-tree, which treats time as a third dimension. In this framework, the MVR-tree facilitates historical single-version access, while the 3DR-tree accelerates cross-version queries. Although these methods provide some level of multi-version access, all updates are restricted to the latest version. Consequently, they do not support advanced Git-style version control, such as branching and what-if analysis [16].

Functional Data Structures. Functional data structures provide a robust foundation for implementing Git-style versioning. The Parallel Augmented Map (PAM) [61] maintains multi-version maps using functional binary search trees with join-based algorithms [12]. The Parallel Compressed tree (PaC-tree) [20] further refines this approach by grouping leaf nodes into compressed blocks, significantly improving space efficiency without compromising update or query throughput. PAM has also been successfully adapted to implement snapshot isolation in multicore in-memory HTAP systems [60]. While these structures are highly efficient for generic key-value data, they do not natively support spatial data indexing. Our work builds upon this functional paradigm, extending these principles to support comprehensive versioned access within a spatial context.

11 Polygon Support

In this section, we discuss the extensibility of SILVA to support complex spatial objects such as polygons. We implement full polygon support for PaC-Z tree within SILVA and compare its index building, updating, and intersection query performance against MVR-tree, MV3R-tree, and boost-R.

Among the three indexes available in SILVA, we extend PaC-Z tree for polygon indexing due to its structural similarity to traditional R-trees: we adopt the classic filter-and-refine approach. Instead of indexing the complex shapes directly, the index stores the minimum bounding rectangles (MBRs) of each polygon. During the query procedure, an initial filtering operation is first executed based on these stored MBRs to quickly prune disjoint objects. Subsequently, a refinement step evaluates the exact geometric boundaries of the remaining candidates to obtain the final precise answers.

To integrate this into our framework, we apply a space-filling curve to the centroids of the MBRs to capture their spatial properties. This transformation allows PaC-Z tree to manage rectangles utilizing the exact same mechanism it uses for multidimensional points. More specifically, we compute the *Z-order* values (i.e., Z-curve) of the rectangle centroids and use them as the fundamental keys in PaC-Z tree.

Build. For tree construction, we first compute the keys (i.e., *Z-values*) for the centroids of all polygons. Then, we sort all bounding rectangles based on these computed keys. Following this, a perfectly balanced binary search tree is constructed from the sorted list to serve as the final index. Note that for each tree node, an augmented bounding box is calculated during the backtracking phase of the recursive construction calls, which is similar to the approach used in R-trees. Furthermore, the bounding boxes of the leaf nodes are explicitly updated to encompass the full spatial extent of the polygon geometries..

Fig. 13 shows the index construction time of PaC-Z in SILVA (running both sequentially and in parallel) alongside two baselines: MVR-tree and MV3R-tree, evaluated on `Uniform` and `Varden` datasets containing 100M objects. Noted that the two baselines only support sequential execution. From Fig. 13(a,b), we observe that PaC-Z in SILVA builds the index orders of magnitude faster than both MVR-tree and MV3R-tree, while also consuming less memory. This trend is highly consistent with the results observed when managing multidimensional points.

Batch Insert/Delete For batch updates, we first sort the rectangles using the same approach employed during tree construction. Subsequently, a recursive partitioning procedure is invoked, starting from the root node. The sorted modification list is dynamically partitioned into two disjoint sets: one containing *Z-value* keys less than the root's key, and the other containing keys greater than the root's key. In cases where two keys are identical, we break ties using their unique rectangle IDs. The partition containing keys smaller than the root is then routed to the left child, while the other partition is routed to the right child. This recursive procedure continues recursively until a leaf node is reached. At this point, a linear merge algorithm is executed to finalize the actual insertions and deletions. Finally, during the backtracking, stored augmented bounding box within each traversed tree node is updated accordingly.

Fig. 14 presents the experimental results for batch insertions and deletions on the synthetic `Uniform` and `Varden` datasets with 100M rectangles. Note that MVR-tree and MV3R-tree can only be executed sequentially. We have the following observations: First, consistent with the point data results shown in Fig. 7, all indexes exhibit similar performance when the number of modified rectangles is small. Under these conditions, the required structural adjustments across all indexes are minimal, resulting in comparable execution times. Second, as the volume of modified rectangles increases, the running time of PaC-Z becomes orders of magnitude faster than that of MVR-tree and MV3R-tree. This significant speedup occurs because PaC-Z processes the entire batch of updates simultaneously. In contrast, the two baseline methods must process updates incrementally, leading to expensive rebalancing and node splits.

Intersects Query. We also implement the intersection query for axis-aligned rectangles. Given a query rectangle, a top-down recursive search is initiated from the root to test for intersections against the bounding box of each traversed tree node. If no intersection is detected, the entire subtree rooted at that node is immediately pruned, as none of its descendants can possibly intersect with the query. Otherwise, the search procedure recursively descends to the child nodes until leaf nodes are reached. Subsequently, all rectangles stored within the visited leaf nodes are scanned to collect the candidate results. Similar to many R-tree-based approaches, an additional post-processing phase (often referred to as the refinement step) can be applied to these candidates to derive the final, exact answers.

Fig. 15 presents the performance results for the intersection queries. Since MVR-tree and MV3R-tree rely on identical underlying structures for querying and thus exhibit comparable performance, we only report the results for MVR-tree. We have the following observations: First, among all the evaluated methods, boost-R demonstrates the best intersection query performance. This superior efficiency occurs because boost-R employs a *quadratic* split strategy, which generates more tightly aligned bounding rectangles that are highly beneficial for spatial queries. Second, PaC-Z exhibits slightly slower query times compared to boost-R. This performance gap arises from the unaligned partitioning inherent to PaC-Z, which inadvertently introduces significant overlap among the bounding boxes of tree nodes. Nevertheless, PaC-Z achieves performance comparable to boost-R when the query output size is less than 10^5. In summary, the query performance of PaC-Z is competitive with boost-R, and both indexes execute queries orders of magnitude faster than MVR-tree.

12 Disk Interactions

Although SILVA is designed primarily as a main-memory multi-version spatial index, we have also developed a disk archiving mechanism. This capability is particularly valuable when certain historical versions require long-term storage, or when the accumulation of versions introduces severe memory pressure. In such scenarios, specific versions can be safely offloaded from memory to disk while preserving their pre-computed structural information. This guarantees that when these versions are subsequently retrieved from disk, no additional computational overhead is incurred to reconstruct the index.

At a high level, our approach involves serializing the internal

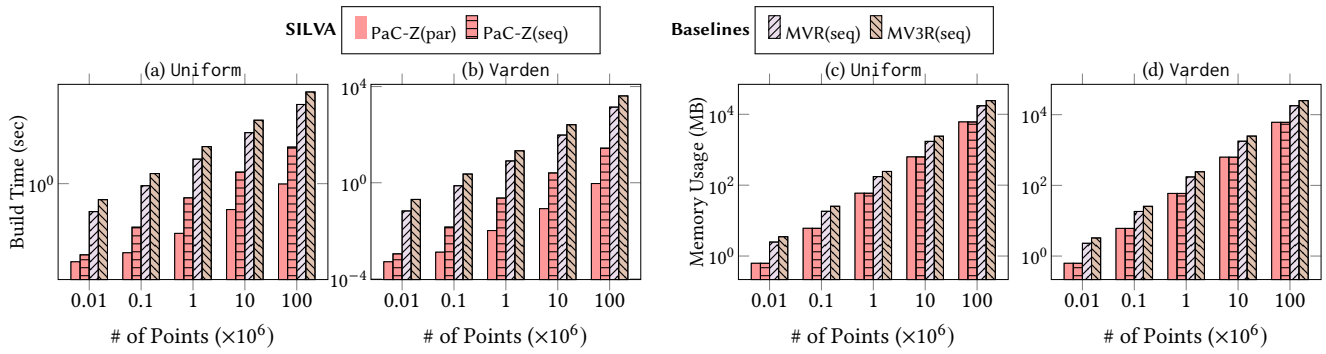

Figure 13: Box Build Time and Memory evaluation on synthetic dataset, lower the better. (a), (b) show the *build time* in *seconds*. (c), (d) show the *Memory Usage* in *Megabytes*. All axes are in *log* scale.

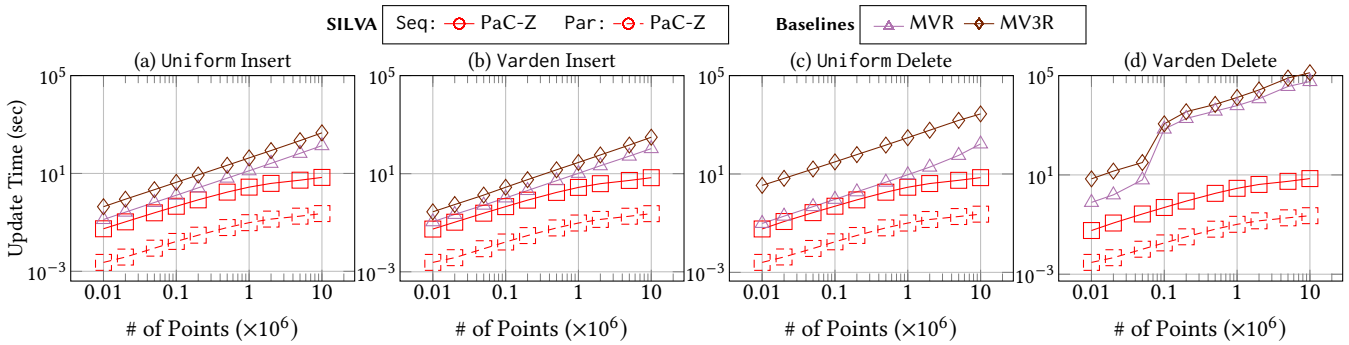

Figure 14: Box Batch Insert/Delete evaluation on synthetic initial version with 100M points, lower the better. (a), (b) show insert time, and (c), (d) show delete time in seconds. All axes are in *log* scale. Solid/Dashed lines represent running in sequential (1 core) and parallel (96 cores), respectively.

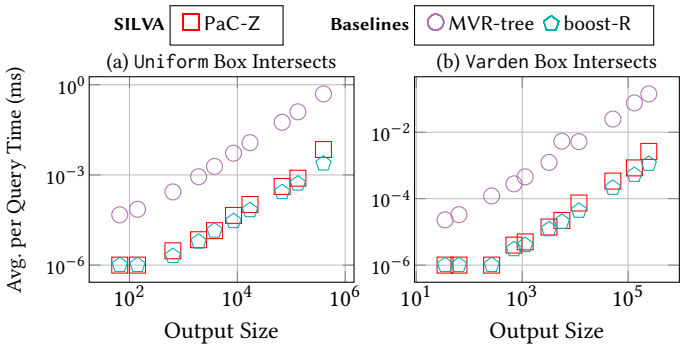

Figure 15: Box intersects query on synthetic datasets, lower the better). All axes are in **log** scale. All queries are read-only and can be parallelized.

index structures to disk and deserializing them upon retrieval.

For MVQ-tree, we perform a breadth-first traversal of the tree, writing the essential data for each visited node to disk. During deserialization, the data is read back into memory, and the corresponding interior and leaf tree nodes are instantiated using the stored information. Consequently, the tree nodes are perfectly reconstructed in the exact same order in which they were originally serialized.

For PaC-Z tree, the serialized entries are stored, and can be read back from disk. These entries are already sorted, and keys and values for each entry are already computed. Therefore, there is no need to recompute them when we loading the index back from disk. When we read them back, a PaC-Z tree can be directly reconstructed based on the entries according to implemented interfaces in PaC-tree [20].

13 Conclusion

In this paper, we propose SILVA, a parallel spatial index library with versioned access using functional data structures. SILVA supports both single- and multi-version spatial data management with high parallelism. SILVA interface facilitates efficient parallel updates that keep historical and branching versions, as well as fundamental version-aware queries such as spatial diff, commit, and merge. Our experiments show that SILVA is consistently faster than existing version-aware spatial indexes in updates, achieving two orders of magnitude faster when running in parallel, while remaining more space-efficient. It also achieves better or comparable query performance, even compared to single-version spatial indexes.

References

[1] J Chris Anderson, Jan Lehnardt, and Noah Slater. 2010. *CouchDB: the definitive guide: time to relax.* " O'Reilly Media, Inc.".

[2] Lars Arge, Klaus H Hinrichs, Jan Vahrenhold, and Jeffrey Scott Vitter. 2002. Efficient bulk operations on dynamic R-trees. *Algorithmica* 33 (2002), 104–128.

[3] Nimar S Arora, Robert D Blumofe, and C Greg Plaxton. 2001. Thread scheduling for multiprogrammed multiprocessors. *Theory of Computing Systems (TOCS)* 34, 2 (2001), 115–144.

[4] Bruno Becker, Stephan Gschwind, Thomas Ohler, Bernhard Seeger, and Peter Widmayer. 1996. An Asymptotically Optimal Multiversion B-Tree. *The VLDB Journal* 5, 4 (1996), 264–275.

[5] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-tree: An efficient and robust access method for points and rectangles. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 322–331.

[6] Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Commun. ACM* 18, 9 (1975), 509–517.

[7] Philip A Bernstein, Colin W Reid, and Sudipto Das. 2011. Hyder-A Transactional Record Manager for Shared Flash.. In *Conference on Innovative Data Systems Research (CIDR)*, Vol. 11. 9–20.

[8] Anant Bhardwaj, Amol Deshpande, Aaron J. Elmore, David Karger, Samuel Madden, Aditya G. Parameswaran, Harihar Subramanyam, Eugene Wu, and Rebecca Zhang. 2015. Collaborative data analytics with DataHub. *Proc. VLDB Endow.* 8, 12 (2015), 1916–1919.

[9] Guy Blelloch, Daniel Ferizovic, and Yihan Sun. 2022. Joinable Parallel Balanced Binary Trees. *ACM Transactions on Parallel Computing (TOPC)* 9, 2 (2022), 1–41.

[10] Guy E. Blelloch, Daniel Anderson, and Laxman Dhulipala. 2020. ParlayLib — a toolkit for parallel algorithms on shared-memory multicore machines. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 507–509.

[11] Guy E Blelloch and Magdalen Dobson. 2022. Parallel Nearest Neighbors in Low Dimensions with Batch Updates. In *Algorithm Engineering and Experiments (ALENEX)*. SIAM, 195–208.

[12] Guy E. Blelloch, Daniel Ferizovic, and Yihan Sun. 2016. Just Join for Parallel Ordered Sets. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*.

[13] Guy E. Blelloch, Jeremy T. Fineman, Yan Gu, and Yihan Sun. 2020. Optimal parallel algorithms in the binary-forking model. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 89–102.

[14] Robert D. Blumofe and Charles E. Leiserson. 1993. Space-efficient scheduling of multithreaded computations. In *ACM Symposium on Theory of Computing (STOC)*. ACM, 362–371.

[15] Boost geometry 2024. Boost Geometry Library. https://www.boost.org/library/latest/geometry/.

[16] Surajit Chaudhuri and Vivek R. Narasayya. 1998. AutoAdmin 'What-if' Index Analysis Utility. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 367–378.

[17] Jack Chen, Samir Jindel, Robert Walzer, Rajkumar Sen, Nika Jimsheleishvilli, and Michael Andrews. 2016. The MemSQL Query Optimizer: A modern optimizer for real-time analytics in a distributed database. *Proceedings of the VLDB Endowment (PVLDB)* 9, 13 (2016), 1401–1412. doi:10.14778/3007263.3007277

[18] Chicago Data Portal 2024. Chicago Crimes: 2001 to Present. https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2/about_data.

[19] Demographic and Economic Data 2022. TIGER with Selected Demographic and Economic Data. https://www.census.gov/geographies/mapping-files/time-series/geo/tiger-data.html.

[20] Laxman Dhulipala, Guy E. Blelloch, Yan Gu, and Yihan Sun. 2022. PaC-trees: Supporting Parallel and Compressed Purely-Functional Collections. In *ACM Conference on Programming Language Design and Implementation (PLDI)*.

[21] Laxman Dhulipala, Guy E Blelloch, and Julian Shun. 2019. Low-latency graph streaming using compressed purely-functional trees. In *ACM Conference on Programming Language Design and Implementation (PLDI)*. 918–934.

[22] Cristian Diaconu, Craig Freedman, Erik Ismert, Per-Ake Larson, Pravin Mittal, Ryan Stonecipher, Nitin Verma, and Mike Zwilling. 2013. Hekaton: SQL server's memory-optimized OLTP engine. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 1243–1254.

[23] Raphael A Finkel and Jon Louis Bentley. 1974. Quad trees a data structure for retrieval on composite keys. *Acta informatica* 4 (1974), 1–9.

[24] Peter Frühwirt, Marcus Huber, Martin Mulazzani, and Edgar R Weippl. 2010. InnoDB database forensics. In *2010 24th IEEE International Conference on Advanced Information Networking and Applications*. IEEE, 1028–1036.

[25] Volker Gaede and Oliver Günther. 1998. Multidimensional Access Methods. *ACM Comput. Surv.* 30, 2 (1998), 170–231.

[26] Junhao Gan and Yufei Tao. 2017. On the hardness and approximation of Euclidean DBSCAN. *ACM Transactions on Database Systems (TODS)* 42, 3 (2017), 1–45.

[27] geogig 2020. GeoGig: Distributed Versioned Spatial Data Editing. https://geogig.org/.

[28] Yan Gu, Zachary Napier, and Yihan Sun. 2022. Analysis of Work-Stealing and Parallel Cache Complexity. In *SIAM Symposium on Algorithmic Principles of Computer Systems (APOCS)*. SIAM, 46–60.

[29] Antonin Guttman. 1984. R-trees: A dynamic index structure for spatial searching. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 47–57.

[30] Silu Huang, Liqi Xu, Jialin Liu, Aaron J. Elmore, and Aditya G. Parameswaran. 2017. OrpheusDB: Bolt-on Versioning for Relational Databases. *Proc. VLDB Endow.* 10, 10 (2017), 1130–1141.

[31] Edwin H. Jacox and Hanan Samet. 2007. Spatial join techniques. *ACM Trans. Database Syst.* (2007).

[32] Ibrahim Kamel and Christos Faloutsos. 1992. Parallel R-trees. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 21, 2 (1992), 195–204.

[33] kart 2024. Kart: Distributed Version Control for Geospatial Data. https://kartproject.org/.

[34] Alfons Kemper and Thomas Neumann. 2011. HyPer: A hybrid OLTP&OLAP main memory database system based on virtual memory snapshots. In *IEEE International Conference on Data Engineering (ICDE)*, Serge Abiteboul, Klemens Böhm, Christoph Koch, and Kian-Lee Tan (Eds.). IEEE Computer Society, 195–206.

[35] Curtis P. Kolovson and Michael Stonebraker. 1991. Segment Indexes: Dynamic Indexing Techniques for Multi-Dimensional Interval Data. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM Press, 138–147.

[36] Per-Åke Larson, Spyros Blanas, Cristian Diaconu, Craig Freedman, Jignesh M Patel, and Mike Zwilling. 2011. High-performance concurrency control mechanisms for main-memory databases. *Proceedings of the VLDB Endowment (PVLDB)* 5, 4 (2011), 298–309.

[37] Scott T. Leutenegger, Jeffrey Edgington, and Mario Alberto López. 1997. STR: A Simple and Efficient Algorithm for R-Tree Packing. In *IEEE International Conference on Data Engineering (ICDE)*. 497–506.

[38] Justin J. Levandoski, David B. Lomet, and Sudipta Sengupta. 2013. The Bw-Tree: A B-tree for New Hardware Platforms. In *IEEE International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 302–313.

[39] libspatialindex 2024. libspatialindex. https://libspatialindex.org/en/latest/.

[40] lightning 2015. Lightning memory-mapped database manager (LMDB). https://dbdb.io/db/lmdb.

[41] Los Angeles Open Data 2024. Traffic Collision Data from 2010 to Present. https://data.lacity.org/Public-Safety/Traffic-Collision-Data-from-2010-to-Present/d5tf-ez2w/about_data.

[42] Michael Maddox, David Goehring, Aaron J. Elmore, Samuel Madden, Aditya G. Parameswaran, and Amol Deshpande. 2016. Decibel: the relational dataset branching system. *Proc. VLDB Endow.* 9, 9 (2016), 624–635.

[43] Ziyang Men, Bo Huang, Yan Gu, and Yihan Sun. 2026. Parallel Dynamic Spatial Indexes. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 150–163.

[44] Ziyang Men, Zheqi Shen, Yan Gu, and Yihan Sun. 2025. Parallel kd-tree with Batch Updates. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 3, 1 (2025), 62:1–62:26.

[45] MicrosoftSQL 2025. Microsoft SQL server. https://www.microsoft.com/en-us/sql-server/.

[46] Guy M Morton. 1966. A computer oriented geodetic data base and a new technique in file sequencing. (1966).

[47] Mario A. Nascimento and Jefferson R. O. Silva. 1998. Towards Historical R-trees. In *ACM symposium on Applied Computing (SAC)*. ACM, 235–240.

[48] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The log-structured merge-tree (LSM-tree). *Acta Inf.* 33, 4 (1996).

[49] Open Street Map Data 2024. Source of OSM data. https://download.geofabrik.de/.

[50] Jack A. Orenstein. 1986. Spatial Query Processing in an Object-Oriented Database System. In *SIGMOD Conference*. ACM Press, 326–336.

[51] Jack A. Orenstein and T. H. Merrett. 1984. A Class of Data Structures for Associative Searching. In *PODS*. ACM, 181–190.

[52] Frank Ramsak, Volker Markl, Robert Fenk, Martin Zirkel, Klaus Elhardt, and Rudolf Bayer. 2000. Integrating the UB-Tree into a Database System Kernel. In *VLDB*. Morgan Kaufmann, 263–272.

[53] Redis 2024. Redis: The Real-Time Data Platform. https://redis.io/.

[54] Simonas Saltenis, Christian S. Jensen, Scott T. Leutenegger, and Mario Alberto López. 2000. Indexing the Positions of Continuously Moving Objects. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM, 331–342.

[55] SILVA Library 2025. Implementation of SILVA: Spatial Index Library with Versioned Access. https://github.com/ItakEjgo/mvzd.

[56] SILVA Library 2025. SILVA: Parallel Spatial Index Library with Versioned Access (Technical Report). https://github.com/ItakEjgo/mvzd/blob/master/SILVA-technical-report.pdf.

[57] spatialhadoop 2025. SpatialHadoop. https://spatialhadoop.cs.umn.edu/.

[58] Michael Stonebraker, Samuel Madden, Daniel J. Abadi, Stavros Harizopoulos, Nabil Hachem, and Pat Helland. 2007. The End of an Architectural Era (It's Time for a Complete Rewrite). In *Proceedings of the VLDB Endowment (PVLDB)*. ACM,

1150–1160.

[59] Yihan Sun and Guy Blelloch. 2019. Implementing Parallel and Concurrent Tree Structures. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 447–450.

[60] Yihan Sun, Guy E Blelloch, Wan Shen Lim, and Andrew Pavlo. 2019. On supporting efficient snapshot isolation for hybrid workloads with multi-versioned indexes. *Proceedings of the VLDB Endowment (PVLDB)* 13, 2 (2019), 211–225.

[61] Yihan Sun, Daniel Ferizovic, and Guy E Blelloch. 2018. PAM: Parallel Augmented Maps. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*.

[62] Yufei Tao and Dimitris Papadias. 2001. MV3R-Tree: A Spatio-Temporal Access Method for Timestamp and Interval Queries. In *Proceedings of the VLDB Endowment (PVLDB)*. Morgan Kaufmann, 431–440.

[63] Yufei Tao, Dimitris Papadias, and Jimeng Sun. 2003. The TPR*-Tree: An Optimized Spatio-Temporal Access Method for Predictive Queries. In *Proceedings of the VLDB Endowment (PVLDB)*. 790–801.

[64] John Woodfill and Michael Stonebraker. 1983. An Implementation of Hypothetical Relations. In *Proceedings of the VLDB Endowment (PVLDB)*. Morgan Kaufmann, 157–166.

[65] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. 2017. An Empirical Evaluation of In-Memory Multi-Version Concurrency Control. *Proceedings of the VLDB Endowment (PVLDB)* 10 (2017), 781–792. Issue 7.